



QB 03: Operating Systems — Question Bank with Answer Keys

Course: Operating Systems

Target: VTU MCA Semester 1 / Semester 2 (PCC)

Scheme: VTU 2022 / 2024 Scheme

Contents: CPU Scheduling Numericals, Process Synchronization, Banker's Deadlock Algorithm, Memory Management, Page Replacement (FIFO, LRU, OPT), 15 MCQs with Explanations



Syllabus Outline (VTU 5 Modules)

- **Module 1:** OS Architecture, Dual-Mode Operations (Kernel vs User), System Calls, Process Management, Process States, PCB, Context Switching, Threads (User vs Kernel), IPC.
- **Module 2:** CPU Scheduling: Scheduling Criteria, Preemptive vs Non-Preemptive, FCFS, SJF, SRTF, Round Robin (RR), Priority Scheduling, Multi-Level Feedback Queues.
- **Module 3:** Process Synchronization: Race Conditions, Critical Section Problem, Peterson's Solution, Semaphores (Binary & Counting), Mutex, Producer-Consumer, Dining Philosophers, Readers-Writers.
- **Module 4:** Deadlocks: 4 Coffman Conditions, Resource Allocation Graph (RAG), Deadlock Prevention, Avoidance (Banker's Algorithm), Detection, and Recovery.
- **Module 5:** Memory Management & Virtual Memory: Contiguous vs Non-Contiguous Allocation, Paging, Page Table, TLB, Segmentation, Demand Paging, Page Replacement Algorithms (FIFO, LRU, Optimal), Thrashing.

SECTION 1: High-Yield MCQs with Answer Key

Q1. Which state transition occurs when a running process initiates an I/O disk request?

- A) Running $\rightarrow$ Ready
- B) Running $\rightarrow$ Terminated
- C) Running $\rightarrow$ Waiting / Blocked
- D) Waiting $\rightarrow$ Ready

Answer: C

Explanation: When a process requests an asynchronous resource or I/O operation, it transitions from Running to Waiting/Blocked until the I/O event completes.

Q2. Which CPU scheduling algorithm is inherently prone to the "Convoy Effect"?

- A) Round Robin

- B) First-Come, First-Served (FCFS)
- C) Shortest Job First (SJF)
- D) Priority Scheduling

Answer: B

Explanation: In FCFS, if a long CPU-bound process arrives first, all short I/O-bound processes are blocked behind it, drastically increasing average waiting time (Convoy effect).

Q3. What is the fundamental requirement satisfied by a solution to the Critical Section problem?

- A) Mutual Exclusion, Progress, Bounded Waiting
- B) High Throughput, Zero Latency, Low Overhead
- C) Multithreading, Starvation, Paging
- D) Preemption, Aging, Deadlock

Answer: A

Explanation: A valid solution to the critical section problem must satisfy three properties: (1) Mutual Exclusion, (2) Progress, and (3) Bounded Waiting.

Q4. Which of the following is NOT one of Coffman's four necessary conditions for deadlock?

- A) Mutual Exclusion
- B) Hold and Wait
- C) Preemption allowed
- D) Circular Wait

Answer: C

Explanation: Deadlock requires **No Preemption** (resources cannot be forcibly confiscated). If preemption is allowed, deadlock cannot occur.

Q5. What is Belady's Anomaly in virtual memory page replacement?

- A) Increasing physical frame allocation decreases page faults
- B) Increasing physical frame allocation leads to MORE page faults
- C) Page fault rate remains constant
- D) LRU performs worse than Random replacement

Answer: B

Explanation: Belady's Anomaly occurs in certain page replacement algorithms (notably **FIFO**), where allocating more page frames to a process unexpectedly increases the total number of page faults.

Q6. Which register in hardware defines the starting physical memory address allocated to a user process?

- A) Limit Register

- B) Base / Relocation Register
- C) Instruction Pointer
- D) Stack Pointer

Answer: B

Explanation: The Base (Relocation) register holds the smallest physical memory address of the process; the Limit register specifies the size/range.

Q7. What is "Thrashing" in an operating system?

- A) A rapid disk write operation
- B) Excessive paging activity where the system spends more time servicing page faults than executing instructions
- C) High CPU utilization during heavy compilation
- D) Immediate termination of background threads

Answer: B

Explanation: Thrashing happens when the sum of working set sizes of all active processes exceeds available physical RAM, causing continuous swapping and collapsing CPU utilization to near zero.

Q8. What does a counting semaphore initialized to value 5 represent?

- A) Up to 5 threads can concurrently access the shared resource
- B) Exactly 5 processes are in deadlock
- C) 5 processors are installed
- D) Binary mutual exclusion with 5 retries

Answer: A

Explanation: A counting semaphore tracks the number of identical available resources. Value 5 permits 5 consecutive `wait()` / `P()` calls without blocking.

Q9. In the Banker's Algorithm, a state is considered "Safe" if:

- A) No process ever requests resources
- B) There exists at least one sequence $\langle P_1, P_2, \dots, P_n \rangle$ in which all processes can satisfy their maximum needs and terminate without deadlock
- C) All resource counts are zero
- D) Preemption is triggered immediately

Answer: B

Explanation: A state is safe if the OS can allocate resources to every process in some safe order such that each process can complete and release its resources.

Q10. What is the role of the Translation Lookaside Buffer (TLB)?

- A) Caches hard drive sectors
- B) High-speed associative hardware cache that speeds up virtual-to-physical address translation
- C) Resolves DNS addresses
- D) Stores CPU registers during context switch

Answer: B

Explanation: TLB is an ultra-fast hardware associative memory that caches recent page table entries (Virtual Page Number to Frame Number) to eliminate extra RAM memory lookups.

**SECTION 2: Core Concept Short Answers (4–6 Marks)****Q11. Compare a Process and a Thread. What is PCB?****Answer:**

| Feature | Process | Thread (Lightweight Process) | |---|---|---| | **Definition** | An executing program with its own isolated address space. | A basic unit of CPU execution within a process. | | **Memory** | Has separate text, data, heap, and open file handles. | Shares text, data, heap, and files with peer threads in the same process. | | **Context Switch** | Heavyweight and slower (flushes TLB, swaps address maps). | Lightweight and fast (shares memory space; only registers and stack change). | | **Crash Impact** | If one process crashes, other processes remain unaffected. | If one thread crashes due to a segfault, the entire parent process crashes. |

Process Control Block (PCB):

A kernel data structure that stores all information needed to manage a process:

- **Process ID (PID) & Process State** (New, Ready, Running, Waiting, Terminated)
- **Program Counter (PC):** Address of next instruction to execute
- **CPU Registers:** Accumulators, index registers, stack pointer
- **Memory Management Information:** Page tables or segment tables
- **Accounting & I/O Status Information:** CPU time used, list of open file descriptors.

Q12. Explain the Producer-Consumer Problem using Semaphores.

Answer: The Producer-Consumer problem involves two processes sharing a common fixed-size buffer of capacity N .

Semaphores used:

- `mutex` : Binary semaphore (initialized to 1) for mutual exclusion while inserting/removing items.
- `empty` : Counting semaphore (initialized to N) counting empty buffer slots.
- `full` : Counting semaphore (initialized to 0) counting filled slots with data items.

```

// Shared variables
semaphore mutex = 1;
semaphore empty = N;
semaphore full = 0;

// Producer Process
void producer() {
    while (1) {
        item = produce_item();
        wait(empty); // Decrement empty slot count (blocks if empty == 0)
        wait(mutex); // Lock buffer

        insert_item(item);

        signal(mutex); // Unlock buffer
        signal(full); // Increment filled slot count
    }
}

// Consumer Process
void consumer() {
    while (1) {
        wait(full); // Decrement filled slot count (blocks if full == 0)
        wait(mutex); // Lock buffer

        item = remove_item();

        signal(mutex); // Unlock buffer
        signal(empty); // Increment empty slot count

        consume_item(item);
    }
}

```

SECTION 3: VTU Model Numerical & Long Questions (10–12 Marks)

Q13. [VTU Model QP - CPU Scheduling Problem]

Consider the following set of 5 processes with arrival times and CPU burst times given in milliseconds:

Process	Arrival Time (\$AT\$)	Burst Time (\$BT\$)	Priority (Lower value = Higher priority)
\$P_1\$	0	8	3
\$P_2\$	1	4	1
\$P_3\$	2	9	4
\$P_4\$	3	5	2
\$P_5\$	4	2	5

Calculate the Average Turnaround Time (TAT) and Average Waiting Time (WT) using: (a) Preemptive Shortest Remaining Time First (SRTF)

(b) Non-preemptive Priority Scheduling

(c) Round Robin (RR) with Time Quantum = 3 ms

Answer:

(a) Preemptive Shortest Remaining Time First (SRTF):

| Process | Need (A B C) | |---|---| | \$P_0\$ | \$(7-0, ; 5-1, ; 3-0) = \mathbf{[7, 4, 3]}\$ | | \$P_1\$ | \$(3-2, ; 2-0, ; 2-0) = \mathbf{[1, 2, 2]}\$ | | \$P_2\$ | \$(9-3, ; 0-0, ; 2-2) = \mathbf{[6, 0, 0]}\$ | | \$P_3\$ | \$(2-2, ; 2-1, ; 2-1) = \mathbf{[0, 1, 1]}\$ | | \$P_4\$ | \$(4-0, ; 3-0, ; 3-2) = \mathbf{[4, 3, 1]}\$ |

Available Vector:

Total allocated:

$$A = 0+2+3+2+0 = 7$$

$$B = 1+0+0+1+0 = 2$$

$$C = 0+0+2+1+2 = 5$$

$$\text{Available} = \text{Total} - \text{Allocated} = [10-7, ; 5-2, ; 7-5] = \mathbf{[3, 3, 2]}$$

(b) Safety Algorithm Execution:

Initial **Work** = [3, 3, 2], **Finish** = [False, False, False, False, False].

1. **Step 1:** Check \$P_0\$: Need \$[7, 4, 3] \le [3, 3, 2]\$? **False** (\$7 > 3\$).
2. **Step 2:** Check \$P_1\$: Need \$[1, 2, 2] \le [3, 3, 2]\$? **True!**
 - \$P_1\$ executes and releases allocation:
 $\text{Work} = \text{Work} + \text{Alloc}_{P_1} = [3, 3, 2] + [2, 0, 0] = \mathbf{[5, 3, 2]}$.
 - \$\text{Finish}[P_1] = \text{True}\$.
3. **Step 3:** Check \$P_3\$: Need \$[0, 1, 1] \le [5, 3, 2]\$? **True!**
 - \$P_3\$ executes and releases allocation:
 $\text{Work} = [5, 3, 2] + [2, 1, 1] = \mathbf{[7, 4, 3]}$.
 - \$\text{Finish}[P_3] = \text{True}\$.
4. **Step 4:** Check \$P_4\$: Need \$[4, 3, 1] \le [7, 4, 3]\$? **True!**
 - \$P_4\$ executes and releases allocation:
 $\text{Work} = [7, 4, 3] + [0, 0, 2] = \mathbf{[7, 4, 5]}$.
 - \$\text{Finish}[P_4] = \text{True}\$.
5. **Step 5:** Check \$P_0\$: Need \$[7, 4, 3] \le [7, 4, 5]\$? **True!**
 - \$P_0\$ executes and releases allocation:
 $\text{Work} = [7, 4, 5] + [0, 1, 0] = \mathbf{[7, 5, 5]}$.
 - \$\text{Finish}[P_0] = \text{True}\$.
6. **Step 6:** Check \$P_2\$: Need \$[6, 0, 0] \le [7, 5, 5]\$? **True!**
 - \$P_2\$ executes and releases allocation:
 $\text{Work} = [7, 5, 5] + [3, 0, 2] = \mathbf{[10, 5, 7]}$ (All resources recovered!).
 - \$\text{Finish}[P_2] = \text{True}\$.

Conclusion: All processes finished! The system is in a **Safe State**.

Safe Sequence: \$\mathbf{\langle P_1, P_3, P_4, P_0, P_2 \rangle}\$ (or \$\langle P_1, P_3, P_0, P_2, P_4 \rangle\$).

(c) Resource-Request for \$P_1 = [1, 0, 2]\$:

1. Is \$\text{Request}_1 \le \text{Need}_1\$?
 $[1, 0, 2] \le [1, 2, 2]$ \$\to\$ **True**.

2. Is $\text{Request}_1 \leq \text{Available}$?

$[1, 0, 2] \leq [3, 3, 2]$ $\rightarrow$ **True**.

3. Pretend allocation:

- $\text{Available}' = [3, 3, 2] - [1, 0, 2] = [2, 3, 0]$
- $\text{Alloc}'_{P_1} = [2, 0, 0] + [1, 0, 2] = [3, 0, 2]$
- $\text{Need}'_{P_1} = [1, 2, 2] - [1, 0, 2] = [0, 2, 0]$

4. Check if new state is safe:

- With **Available = [2, 3, 0]**, P_1 needs $[0, 2, 0] \leq [2, 3, 0]$ $\rightarrow$ Runs!
- $\text{Work} = [2, 3, 0] + [3, 0, 2] = [5, 3, 2]$.
- From $[5, 3, 2]$, sequence $\langle P_3, P_4, P_0, P_2 \rangle$ succeeds as proven before. **Conclusion:** Yes, the request of P_1 can be **granted immediately** without causing deadlock.

Q15. [VTU Model QP - Page Replacement Algorithms]

A virtual memory system has 3 physical page frames. Consider the following reference string:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1

Calculate the number of Page Faults and Hit Ratio using:

(a) FIFO (First-In, First-Out)

(b) LRU (Least Recently Used)

(c) Optimal (OPT) Page Replacement

Answer:

(a) FIFO (First-In, First-Out) — 3 Frames:

- 7 : Miss (7, -, -) $\rightarrow$ Fault 1
- 0 : Miss (7, 0, -) $\rightarrow$ Fault 2
- 1 : Miss (7, 0, 1) $\rightarrow$ Fault 3
- 2 : Miss (2 replaces 7) $\rightarrow$ Fault 4
- 0 : Hit (2, 0, 1)
- 3 : Miss (3 replaces 0) $\rightarrow$ Fault 5
- 0 : Miss (0 replaces 1) $\rightarrow$ Fault 6
- 4 : Miss (4 replaces 2) $\rightarrow$ Fault 7
- 2 : Miss (2 replaces 3) $\rightarrow$ Fault 8
- 3 : Miss (3 replaces 0) $\rightarrow$ Fault 9
- 0 : Miss (0 replaces 4) $\rightarrow$ Fault 10
- 3 : Hit (0, 2, 3)
- 2 : Hit (0, 2, 3)
- 1 : Miss (1 replaces 2) $\rightarrow$ Fault 11
- 2 : Miss (2 replaces 3) $\rightarrow$ Fault 12

- 0 : Hit (0, 1, 2)
- 1 : Hit (0, 1, 2)
- 7 : Miss (7 replaces 0) $\rightarrow$ Fault 13
- 0 : Miss (0 replaces 1) $\rightarrow$ Fault 14
- 1 : Miss (1 replaces 2) $\rightarrow$ Fault 15

Total Page Faults (FIFO): 15

Hits: 5

Hit Ratio: $\frac{5}{20} = \mathbf{25\%}$

(b) LRU (Least Recently Used) — 3 Frames:

- Tracks the page unreferenced for the longest past time.
- Total Faults = **12**
- Hits = **8**
- **Hit Ratio:** $\frac{8}{20} = \mathbf{40\%}$

(c) Optimal (OPT) — 3 Frames:

- Replaces the page that will NOT be used for the longest future time.
- Total Faults = **9**
- Hits = **11**
- **Hit Ratio:** $\frac{11}{20} = \mathbf{55\%}$

Summary Comparison Table: | Algorithm | Page Faults | Hits | Hit Ratio | Belady's Anomaly? | |---|---|---|---|---| |
FIFO | 15 | 5 | 25.0% | **Yes** (Subject to anomaly) | | **LRU** | 12 | 8 | 40.0% | **No** (Stack algorithm) | | **Optimal** | 9 | 11 | 55.0% | **No** (Theoretical benchmark) |

Key Takeaway Checklist for Exam Day

- [] Practice drawing clear Gantt charts for CPU scheduling. Label each slice with process ID and start/finish timestamps.
- [] For Banker's algorithm, double check: $\text{Available} = \text{Total Instances} - \sum \text{Allocations}$.
- [] In page replacement tables, mark Page Hits with asterisks ***** or checkmarks $\checkmark$ for evaluators.